

Introduction to Tensors with Tensorflow

Hello everyone, It's been a very long time since I've made any articles and I've been kinda missing the fun of writing articles. So, I'm back with another article series. This article series will be about `Neural Networks` and `TensorFlow`. As always I'm `Md. Rishat Talukder AKA Itvaya`. let's get this thing started.

- [LinkedIn](#)
- [YouTube](#)
- [github](#)
- [Gmail](#)
- [discord](#)

prerequisite:

- **Good understanding of python.**
- **Good understanding of numpy and pandas.**
- **Optional: Overall basic understanding of machine learning.**

Introduction to Tensorflow

What is `TensorFlow` ?

`TensorFlow` is an open-source machine learning library for research and production. TensorFlow offers APIs for beginners and experts to develop for desktop, mobile, web, and cloud. See the sections below to get started.

I like to think of tensorflow as a more powerful version of `numpy`. If you know numpy your work is already 60% done.

In this article we will see how to get started with `TensorFlow` and what are `tensors` ?

Installation

```
python3 -m pip install 'tensorflow[and-cuda]'
```

If you have a Nvidia GPU this should work amazing... And if you don't have a gpu there should be no issue there too.

```
In [1]: import warnings

warnings.filterwarnings("ignore")
```

```
In [2]: import tensorflow as tf
print(tf.__version__)
```

```
2026-06-02 13:09:28.368708: I tensorflow/core/platform/cpu_feature_guard.c
c:210] This TensorFlow binary is optimized to use available CPU instructio
ns in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebui
ld TensorFlow with the appropriate compiler flags.
2.20.0
```

There you go we have `imported tensorflow` and printed the version of the `tensorflow` library. I'm using `TensorFlow 2.16.1` in this notebook.

Tensors

A `tensor` is a `generalization` of vectors and matrices to potentially higher dimensions. Internally, TensorFlow represents tensors as n-dimensional arrays of base datatypes.

there many ways to create `tensors` in `tensorflow` but most common ways are using `tf.constant` and `tf.Variable` functions.

tf.constant

Here is how you can create a `tensor` using `tf.constant` function

```
In [3]: scaler = tf.constant(1)
        scaler
```

```
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1780384183.754953    82423 gpu_device.cc:2020] Created device /
job:localhost/replica:0/task:0/device:GPU:0 with 9382 MB memory:  -> devic
e: 0, name: NVIDIA GeForce RTX 3060, pci bus id: 0000:05:00.0, compute cap
ability: 8.6
```

```
Out[3]: <tf.Tensor: shape=(), dtype=int32, numpy=1>
```

here, we have created a `tensor` that contains a numpy value `1` and the datatype of the tensor is `int32`.

if you take a closer look you can see that the shape of the tensor is `()` which means it is a `0-D` tensor. So, it's a scalar.

We have a tensor in our hand.

Now you might think why does I named the tensor as `scaler` because it's a `0-D` tensor AKA a `scaler`.

A `scaler` value is something that has only magnitude and no direction.

To prove it we can see the dimension of the tensor.

```
In [4]: scaler.ndim
```

```
Out[4]: 0
```

Here you go. You can just use the `variable_name.ndim` to get the dimension of the tensor.

Now, let's see how to create a `1-D` tensor.

```
In [5]: vector = tf.constant([1, 2, 3])  
  
vector
```

```
Out[5]: <tf.Tensor: shape=(3,), dtype=int32, numpy=array([1, 2, 3], dtype=int32)  
>
```

And we have created a `1-D` tensor. We just pass a `list` of values or a `numpy array` to the `tf.constant` function.

And you can see that the tensor is containing a `numpy array` of values `[1, 2, 3]` and the datatype of the tensor is `int32` and the shape of the tensor is `(3,)` which means it is a `1-D` tensor.

Now we can see the dimensions.

```
In [6]: vector.ndim
```

```
Out[6]: 1
```

and you see that the tensor is a `1-D` tensor.

A `vector` is a `1-D` array of values.

And we can keep going. So let's try to make a `2-D` tensor and also a `3-D` tensor.

```
In [7]: matrix = tf.constant([  
    [1, 2],  
    [3, 4]  
])
```

```
In [8]: matrix.ndim
```

```
Out[8]: 2
```

AAAND we made a `2-D` tensor. We just passed a `list of lists` to the `tf.constant` function. Now we see the shape of the tensor.

```
In [9]: matrix.shape
```

```
Out[9]: TensorShape([2, 2])
```

it's a `2 by 2` tensor. Simply it's a `2-D` array of values is referred to as a `matrix`.

```
In [10]: n_dimantional_matrix = tf.constant([  
    [  
        [1, 2, 3],  
        [4, 5, 6]  
    ],  
    ]
```

```
[
    [7, 8, 9],
    [10, 11, 12]
]
```

```
In [11]: n_dimantional_matrix
```

```
Out[11]: <tf.Tensor: shape=(2, 2, 3), dtype=int32, numpy=
array([[[ 1,  2,  3],
        [ 4,  5,  6]],

       [[ 7,  8,  9],
        [10, 11, 12]]], dtype=int32)>
```

Then you go, we have created a 3-D tensor. We just passed a list of lists of lists to the `tf.constant` function. Now we see the shape of the tensor.

So, we have created scalar, vector, matrix, and 3-D or anything greater than that is called a tensor in general.

0-d array is called a scalar. 1-d array is called a vector. 2-d array is called a matrix. n-d array is called a tensor.

Now let's see how to create a tensor using the `tf.Variable` function.

`tf.constant` has many more methods and attributes that we can use and also we can pass different arguments to set the data type of the tensor, shape of the tensor, etc.

```
In [12]: tf.constant([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]], dtype=tf
```

```
Out[12]: <tf.Tensor: shape=(2, 2, 3), dtype=float16, numpy=
array([[[ 1.,  2.,  3.],
        [ 4.,  5.,  6.]],

       [[ 7.,  8.,  9.],
        [10., 11., 12.]]], dtype=float16)>
```

YOU should check out the official documentations for [constant](#)

Although `tf.constant` is great for creating a tensor but it's not the only way and this constant tensor is immutable. Once you create a constant tensor you can't change the values of the tensor.

```
In [13]: vector = tf.constant([1,2,3,4,5])
```

```
In [15]: vector[1] = 3
vector
```

```

-----
-
TypeError                                Traceback (most recent call las
t)
Cell In[15], line 1
----> 1 vector[1] = 3
      2 vector

TypeError: 'tensorflow.python.framework.ops.EagerTensor' object does not s
upport item assignment

```

As you can see that you cannot assign a to a index of the vector. It's immutable.

tf.Variable

So, if you want to create a mutable tensor you can use the `tf.Variable` function.

This is a more powerful version of the `tf.constant` function.

Using `tf.Variable` function we can do everything that we can do with `tf.constant` function and many more.

```
In [16]: vector = tf.Variable([1,2,3,4,5])
vector
```

```
Out[16]: <tf.Variable 'Variable:0' shape=(5,) dtype=int32, numpy=array([1, 2, 3,
4, 5], dtype=int32)>
```

```
In [17]: vector.dtype
```

```
Out[17]: tf.int32
```

```
In [18]: vector.shape
```

```
Out[18]: TensorShape([5])
```

Now, let's try to assign a value to a index of the vector.

```
In [19]: vector[1] = 3
```

```

-----
-
TypeError                                Traceback (most recent call las
t)
Cell In[19], line 1
----> 1 vector[1] = 3

TypeError: 'ResourceVariable' object does not support item assignment

```

Annnnd this doesn't work. In tensorflow, assignment doesn't work like normal assignment. You can't use the assignment operator `=` to assign a value to a index of the vector, instead you have to use the `assign` method.

```
In [20]: vector[1].assign(32)
vector
```

```
Out[20]: <tf.Variable 'Variable:0' shape=(5,) dtype=int32, numpy=array([ 1, 32, 3, 4, 5], dtype=int32)>
```

Assignment is working now. Now, before we move to more wierd things I want to talk about the output of the `tf.Variable` function.

```
<tf.Variable 'Variable:0' shape=(5,) dtype=int32,
numpy=array([ 1, 32, 3, 4, 5], dtype=int32)>
```

- The first part is the type of tensor `tf.Variable`.
- `Variable:0` means that the variable is the zeroth variable. Tensorflow assigns unique names to tensor which is made with `tf.Variable` function.
- `shape=(5,)` means that the shape of the tensor is 5.
- `dtype=int32` means that the data type of the tensor is `int32`.
- `numpy=array([ 1, 32, 3, 4, 5], dtype=int32)` means that the values of the tensor are `[ 1, 32, 3, 4, 5]` and the data type of the values is `int32`.

Now if you go to the [official documentations](#) you can see that there are a lot a of arguments that you can pass to the `tf.Variable` function and also a lot of methods and attributes that you can use when you are working with the `tf.Variable` function.

You have access to methods like:

- `assign` to assign a value to a index of the vector.
- `assign_add` to add a value to a index of the vector.
- `assign_sub` to subtract a value from a index of the vector.

Random Tensors

As I said earlier, tensorflow is a powerful and feature rich version of numpy. It can also help us generate random tensors more easily.

```
In [21]: random_1 = tf.random.Generator.from_seed(42)
random_1.normal(shape=(3, 2))
```

```
Out[21]: <tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[ -0.7565803 , -0.06854702],
       [ 0.07595026, -1.2573844 ],
       [-0.23193765, -1.8107855 ]], dtype=float32)>
```

To generate random numbers we can use the `tf.random.Generator` class and start the random number generator with the `from_seed` method.

This will return a `Generator` object that we can use to generate random numbers.

The generator object is like a random number generator with different methods and attributes that we can use to generate random numbers from different distributions like the above I'm using the `normal` method to generate random numbers from a normal distribution with

the shape of `(3, 2)`. Meaning this should generate 3 rows and 2 columns of random numbers.

`from_seed` method takes a `seed` argument which is an integer.

If you don't know that the seed value does here's a short explanation.

We are generating random numbers but these are not pure random values, these are `pseudo random numbers`. Meaning each random number that is generated is dependent on the `seed` value that we pass to the `from_seed` method.

Seed defines the random order of the random numbers that are generated.

So, if we set the seed, the same random number will keep being generated. If we don't then the random order will keep on changing and we will get different random numbers each time we run a file.

It is a best practice to set the seed value to a fixed value so that the random numbers are always the same during production because if the number keeps on changing any bug will be very hard to reproduce.

Again, if you haven't learned numpy or worked with numpy you should learn numpy now, because almost everything that you are doing with tensorflow is based on numpy.

Shuffling the data

In the process of training a machine learning model or a neural network the `order of data matters a lot`. A model can easily get overfitted (learning a pattern so good that learning other patterns becomes tough) so, shuffling the data before training is a must.

To shuffle the data we can use the `tf.random.shuffle` function.

```
In [22]: data = random_1.normal(shape=(5, 3))
data
```

```
Out[22]: <tf.Tensor: shape=(5, 3), dtype=float32, numpy=
array([[ 0.17522676,  0.71105534,  0.5488243 ],
       [ 0.14896014, -0.54757965,  0.61634356],
       [-0.98912024,  2.0912669 ,  0.2183963 ],
       [ 0.93540764, -0.80788165,  0.03395548],
       [-0.6375404 , -0.8360651 ,  0.9296424 ]], dtype=float32)>
```

```
In [23]: tf.random.shuffle(data, seed=42)
```

```
Out[23]: <tf.Tensor: shape=(5, 3), dtype=float32, numpy=
array([[ -0.98912024,  2.0912669 ,  0.2183963 ],
       [ 0.14896014, -0.54757965,  0.61634356],
       [ 0.93540764, -0.80788165,  0.03395548],
       [-0.6375404 , -0.8360651 ,  0.9296424 ],
       [ 0.17522676,  0.71105534,  0.5488243 ]], dtype=float32)>
```

There are other methods like `tf.random.stateless_shuffle` that can be used to shuffle the data but I think it is better to use `tf.random.shuffle` because it is more performant. But main gist is we are suffling the data before training.

You guys can use the `tf.random` module to generate different types of random numbers with different distributions and also suffle pre-existing data in different ways.

We are big boys now, go take a look at the documentations [here](#)

Numpy vs Tensorflow

Numpy is an amazing library for mathematics and data manipulation and the fun thing about tensorflow is that it is a more powerful and feature rich version of numpy.

Internally almost everything that you are doing with tensorflow is based on numpy.

It is directly compatible with numpy arrays and you get many of the same methods and attributes that you can use with numpy arrays.

Ones and Zeros

In numpy you can use the `np.ones` and `np.zeros` functions to create tensors.

```
In [24]: import numpy as np  
  
np.ones((3, 4))
```

```
Out[24]: array([[1., 1., 1., 1.],  
               [1., 1., 1., 1.],  
               [1., 1., 1., 1.]])
```

In tensorflow you can use the `tf.ones` and `tf.zeros` functions to create tensors.

```
In [25]: tf.ones((3, 4))  
  
Out[25]: <tf.Tensor: shape=(3, 4), dtype=float32, numpy=  
array([[1., 1., 1., 1.],  
       [1., 1., 1., 1.],  
       [1., 1., 1., 1.]], dtype=float32)>
```

We can set the shape of the tensor using the `shape` argument, just like we did in numpy.

```
In [26]: np.zeros(shape=(3, 4))  
  
Out[26]: array([[0., 0., 0., 0.],  
               [0., 0., 0., 0.],  
               [0., 0., 0., 0.]])
```

```
In [27]: tf.zeros(shape=(3, 4))
```

```
Out[27]: <tf.Tensor: shape=(3, 4), dtype=float32, numpy=
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]], dtype=float32)>
```

Converting numpy arrays to tensors

Another cool part is that we can make a numpy array in numpy and then convert it to a tensor in tensorflow. I do that most of the time.

```
In [28]: numpy_arr = np.arange(1, 25, dtype=np.int32)
numpy_arr
```

```
Out[28]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
                18, 19, 20, 21, 22, 23, 24], dtype=int32)
```

Now, we can use both `tf.constant` and `tf.Variable` functions to convert the numpy array to a tensor.

```
In [29]: tf.constant(numpy_arr)
```

```
Out[29]: <tf.Tensor: shape=(24,), dtype=int32, numpy=
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
                18, 19, 20, 21, 22, 23, 24], dtype=int32)>
```

We can set custom shapes for the array using the `shape` argument.

```
In [32]: tensor = tf.constant(numpy_arr, shape=(2, 3, 4))
tensor
```

```
Out[32]: <tf.Tensor: shape=(2, 3, 4), dtype=int32, numpy=
array([[[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]],
       [[13, 14, 15, 16],
        [17, 18, 19, 20],
        [21, 22, 23, 24]]], dtype=int32)>
```

One thing you must remember the shape cannot indicate a bigger or smaller size than the size of the array.

```
In [31]: numpy_arr.size
```

```
Out[31]: 24
```

As you can see I created an array of size 24 and I set the shape to `(2, 3, 4)`.

How, do you figure out what shape to use?

simple, just multiple the shapes together.

$$2 \times 3 \times 4 = 24$$

Main point is that the shape multiplication must be equal to the size of the array.

Getting informations of a tensor

In tensorflow every tensor is filled a lot of informations like the `dtype`, `shape`, `dimensions`, etc.

```
In [35]: tensor = tf.ones(shape=(2, 3, 4))
         tensor

Out[35]: <tf.Tensor: shape=(2, 3, 4), dtype=float32, numpy=
         array([[[1., 1., 1., 1.],
                  [1., 1., 1., 1.],
                  [1., 1., 1., 1.]],
                [[1., 1., 1., 1.],
                  [1., 1., 1., 1.],
                  [1., 1., 1., 1.]]], dtype=float32)>
```

As you can see this is a `3-D` tensor. And the object is returns gives us a lot information too.

But you can also extract these informations from a tensor by it's `attributes`.

```
In [37]: print("Data type of the tensor:", tensor.dtype)
         print("Shape of the tensor:", tensor.shape)
         print("Number of dimensions of the tensor:", tensor.ndim)
         print("Total number of elements in the tensor:", tf.size(tensor))

Data type of the tensor: <dtype: 'float32'>
Shape of the tensor: (2, 3, 4)
Number of dimensions of the tensor: 3
Total number of elements in the tensor: tf.Tensor(24, shape=(), dtype=int32)
```

As you can see the `dtype` is `float32` and the `shape` is `(2, 3, 4)` and the `ndim` is `3` and the `total size` is `24`.

Indexing and Slicing

Now, time for the fun part.

I think I've established that a tensor is a n-dim array and it is almost like numpy arrays. So, it should by default mean the indexing a tensor is the same as indexing a numpy array.

Let's me show you.

```
In [38]: np_arr = np.random.randint(0, 100, size=(4, 4,4))

In [40]: tensor = tf.constant(np_arr)
```

```
tensor
```

```
Out[40]: <tf.Tensor: shape=(4, 4, 4), dtype=int64, numpy=
array([[[44, 65, 68, 99],
        [37, 63, 58, 96],
        [26, 29, 50, 38],
        [ 2, 38, 67, 93]],

       [[ 0,  3, 42, 29],
        [ 8, 29,  8, 52],
        [83, 14, 57, 99],
        [38, 30, 72, 16]],

       [[76, 94, 89, 85],
        [74, 85, 67, 27],
        [96, 10, 76, 12],
        [58, 15, 63, 79]],

       [[30, 55, 53, 54],
        [88, 34, 25, 55],
        [40,  2, 56, 44],
        [57, 91,  6, 14]])]>
```

Here, you can see that the tensor is a 3-D tensor and the shape of the tensor is (4, 4, 4) .

So, what about getting 44 .

```
In [45]: tensor[0][0][0]
```

```
Out[45]: <tf.Tensor: shape=(), dtype=int64, numpy=44>
```

We, have to first get the 0 index of the first dimension and then get the 0 index of the second dimension and then get the 0 index of the third dimension and we get 44 .

Normal indexing.

```
In [46]: tensor[0,0,0]
```

```
Out[46]: <tf.Tensor: shape=(), dtype=int64, numpy=44>
```

ORRRR, we can use the numpy approach and use , to separate the indexes and still this will work the same.

PS: IF YOU KNOW NUMPY YOU BY DEFAULT KNOW THIS.

So, what about slicing?

Slicing is done in the same way as numpy arrays completely.

Let's say I want to get the middle 2 rows and column from all the elements in the 3rd dimension.

```
In [49]: tensor[:, 1:3, 1:3]
```

```
Out[49]: <tf.Tensor: shape=(4, 2, 2), dtype=int64, numpy=
array([[[63, 58],
        [29, 50]],

       [[29,  8],
        [14, 57]],

       [[85, 67],
        [10, 76]],

       [[34, 25],
        [ 2, 56]])>
```

PS: just remember while slicing dimension start from left to right. So, as I want every element of the 3rd dimension I use `:`, then I want the middle 2 rows of the 2nd dimension I use `1:3` and then I want the middle 2 columns of the 1st dimension I use `1:3`.

rows are the second dimension and columns are the first dimension.

Play with this a little and you'll get the hang of it.

Just put a colon in the middle of the dimensions and you'll get all the rows but only the middle columns.

PS: The negative indexing is also supported.

New Axis

If you want to add a new axis to a tensor you can use the `tf.newaxis` attribute. This is useful when you want to add a new dimension to a tensor.

```
In [58]: tensor = tensor[..., tf.newaxis]
         tensor
```

```
Out[58]: <tf.Tensor: shape=(1, 4, 4, 4, 1), dtype=int64, numpy=
array([[[[44],
        [65],
        [68],
        [99]],

       [[37],
        [63],
        [58],
        [96]],

       [[26],
        [29],
        [50],
        [38]],

       [[ 2],
        [38],
        [67],
        [93]]],

      [[[ 0],
        [ 3],
        [42],
        [29]],

       [[ 8],
        [29],
        [ 8],
        [52]],

       [[83],
        [14],
        [57],
        [99]],

       [[38],
        [30],
        [72],
        [16]]],

      [[[76],
        [94],
        [89],
        [85]],

       [[74],
        [85],
        [67],
        [27]],

       [[96],
        [10],
        [76],
        [12]],

       [[58],
        [15],
```

```

        [63],
        [79]]],

[[[30],
  [55],
  [53],
  [54]],

 [[88],
  [34],
  [25],
  [55]],

 [[40],
  [ 2],
  [56],
  [44]],

 [[57],
  [91],
  [ 6],
  [14]]]]])>

```

`...` operator is used to say all the other dimensions. It does the same thing as `tensor[:, :, :, tf.newaxis]` but it is more readable.

So, instead of writing `tensor[:, :, :, tf.newaxis]` you can just write `tensor[..., tf.newaxis]`.

Now, by the shape of the tensor you can see that the `new axis` is added to the last dimension and this might be a little confusing because the tensor looks a little freaky. But what happened is that the `1st dimension` values are now `2nd dimension` values that's why you should see that every value is now inside a array.

PS: here, `tf.newaxis` elevates each dimension to 1 dimension above.

We can do the exact opposite by using the `tf.squeeze` function.

```
In [59]: tf.squeeze(tensor)
```

```
Out[59]: <tf.Tensor: shape=(4, 4, 4), dtype=int64, numpy=
array([[[44, 65, 68, 99],
        [37, 63, 58, 96],
        [26, 29, 50, 38],
        [ 2, 38, 67, 93]],

       [[ 0,  3, 42, 29],
        [ 8, 29,  8, 52],
        [83, 14, 57, 99],
        [38, 30, 72, 16]],

       [[76, 94, 89, 85],
        [74, 85, 67, 27],
        [96, 10, 76, 12],
        [58, 15, 63, 79]],

       [[30, 55, 53, 54],
        [88, 34, 25, 55],
        [40,  2, 56, 44],
        [57, 91,  6, 14]])])>
```

But this is not permanent.

```
In [61]: tensor
```

```
Out[61]: <tf.Tensor: shape=(1, 4, 4, 4, 1), dtype=int64, numpy=
array([[[[44],
        [65],
        [68],
        [99]],

       [[37],
        [63],
        [58],
        [96]],

       [[26],
        [29],
        [50],
        [38]],

       [[ 2],
        [38],
        [67],
        [93]]],

      [[[ 0],
        [ 3],
        [42],
        [29]],

       [[ 8],
        [29],
        [ 8],
        [52]],

       [[83],
        [14],
        [57],
        [99]],

       [[38],
        [30],
        [72],
        [16]]],

      [[[76],
        [94],
        [89],
        [85]],

       [[74],
        [85],
        [67],
        [27]],

       [[96],
        [10],
        [76],
        [12]],

       [[58],
        [15],
```

```

        [63],
        [79]]],

[[[30],
  [55],
  [53],
  [54]],

 [[88],
  [34],
  [25],
  [55]],

 [[40],
  [ 2],
  [56],
  [44]],

 [[57],
  [91],
  [ 6],
  [14]]]]])>

```

see nothing changed.

You task:

- Use the `tf.newaxis` but this time at the start `[tf.newaxis, ...]`
- Also, read documentations to figure out how to add a new axis at the middle of at any dimensions.

Expand_dims

There another way you can add an axis to the tensor is using the `tf.expand_dims` function.

```
In [62]: tensor = tf.squeeze(tensor)
```

```
In [65]: tf.expand_dims(tensor, axis=1)
```

```
Out[65]: <tf.Tensor: shape=(4, 1, 4, 4), dtype=int64, numpy=
array([[[[44, 65, 68, 99],
        [37, 63, 58, 96],
        [26, 29, 50, 38],
        [ 2, 38, 67, 93]]],

       [[ 0,  3, 42, 29],
        [ 8, 29,  8, 52],
        [83, 14, 57, 99],
        [38, 30, 72, 16]]],

       [[76, 94, 89, 85],
        [74, 85, 67, 27],
        [96, 10, 76, 12],
        [58, 15, 63, 79]]],

       [[30, 55, 53, 54],
        [88, 34, 25, 55],
        [40,  2, 56, 44],
        [57, 91,  6, 14]]]])>
```

`expand_dims` method takes two arguments.

- The first argument is the tensor you want to add an axis to.
- The second argument is the axis you want to add the axis to.

Here, I said I want to add a new axis in place of the `1` axis.

So, the shape from (4,4,4) became (4, 1, 4, 4).

This confused me a lot when I was learning. Here's a simple way to understand it.

Think of the shape as an array (4,4,4). Here, each index represents the size of the corresponding dimension. 0 index has 4 elements 1 index has 4 elements and so on.

These indices are referred to as the `axes` of the tensor. So, when I want to add an axis I need to specify which `axis` I want to add the axis to.

So, I put 1 and then in the result I got (4, 1, 4, 4).

Best practice is to follow only 1 approach either use `dimensions` to figure out what's happening or use `axis` to figure out what's happening. I refer to axis because it keeps my brain from saying, `what the meaning of this life at all`.

Tensor Operations

We can do general tensor operations like addition, subtraction, multiplication, etc and like numpy arrays the values of a tensor are broadcasted automatically.

For example.

```
In [69]: tensor + 100
```

```
Out[69]: <tf.Tensor: shape=(4, 4, 4), dtype=int64, numpy=
array([[[144, 165, 168, 199],
        [137, 163, 158, 196],
        [126, 129, 150, 138],
        [102, 138, 167, 193]],

       [[100, 103, 142, 129],
        [108, 129, 108, 152],
        [183, 114, 157, 199],
        [138, 130, 172, 116]],

       [[176, 194, 189, 185],
        [174, 185, 167, 127],
        [196, 110, 176, 112],
        [158, 115, 163, 179]],

       [[130, 155, 153, 154],
        [188, 134, 125, 155],
        [140, 102, 156, 144],
        [157, 191, 106, 114]])]>
```

As you can see, all the values of the tensor are added by 100. This is called broadcasting. So, when we are doing an operation in a tensor the operation is applied to all the elements of the tensor.

So, we can do operations like

```
In [71]: # print(tensor + 100)
# print(tensor - 100)
# print(tensor * 100)
# print(tensor / 100)
```

Another special operation is the `@` operator. This is called matrix multiplication.

```
In [76]: tensor1 = tf.constant(np.random.randint(0, 100, size=(2, 3)))
tensor2 = tf.constant(np.random.randint(0, 100, size=(3, 2)))

tensor1, tensor2
```

```
Out[76]: (<tf.Tensor: shape=(2, 3), dtype=int64, numpy=
array([[72, 76, 18],
       [55, 80, 27]])>,
<tf.Tensor: shape=(3, 2), dtype=int64, numpy=
array([[84, 11],
       [93, 18],
       [84, 27]])>)
```

Now we can do matrix multiplication.

```
In [77]: tensor1 @ tensor2
```

```
Out[77]: <tf.Tensor: shape=(2, 2), dtype=int64, numpy=
array([[14628, 2646],
       [14328, 2774]])>
```

Rules for matrix multiplication:

- Inner dimensions must match.

If you don't know what matrix multiplication is or how to do it, [here](#) (https://www.khanacademy.org/multivariable-calculus/multivariable-calculus/a/matrix-multiplication-is-not-matrix-multiplication/a/matrix-multiplication-is-not-matrix-multiplication) is great resource.

Also there are matrix play grounds where you can learn more about matrix interectively.

[Matrix Playground](#)

Some more built in operations and functions

You can do a lot of operations and functions in tensorflow. Here are some of them.

PS: as every tensor is internally a numpy array you can extract the numpy array from the tensor using the `.numpy()` method.

```
In [85]: print("--- Information Functions ---")
# 1. Get shape, data type, and total number of elements
print("Shape of tensor:", tensor.shape)
print("Data type:", tensor.dtype)
print("Total elements:", tf.size(tensor).numpy())
```

```
--- Information Functions ---
Shape of tensor: (4, 4, 4)
Data type: <dtype: 'int64'>
Total elements: 64
```

```
In [80]: print("\n--- Value Aggregation ---")
# 2. Global Minimum and Maximum values
print("Global Min:", tf.reduce_min(tensor).numpy())
print("Global Max:", tf.reduce_max(tensor).numpy())
```

```
--- Value Aggregation ---
Global Min: 0
Global Max: 99
```

```
In [87]: print("\n--- Axis-Specific Aggregation ---")
# 3. Min/Max along columns (axis=0)
print("Min of each column:")
print(tf.reduce_min(tensor, axis=0).numpy())
print("Max of each column:")
print(tf.reduce_max(tensor, axis=0).numpy())
```

--- Axis-Specific Aggregation ---

Min of each column:

```
[[ 0  3 42 29]
 [ 8 29  8 27]
 [26  2 50 12]
 [ 2 15  6 14]]
```

Max of each column:

```
[[76 94 89 99]
 [88 85 67 96]
 [96 29 76 99]
 [58 91 72 93]]
```

```
In [88]: # 4. Min/Max along rows (axis=1)
print("Min of each row:")
print(tf.reduce_min(tensor, axis=1).numpy())
print("Max of each row:")
print(tf.reduce_max(tensor, axis=1).numpy())
```

Min of each row:

```
[[ 2 29 50 38]
 [ 0  3  8 16]
 [58 10 63 12]
 [30  2  6 14]]
```

Max of each row:

```
[[44 65 68 99]
 [83 30 72 99]
 [96 94 89 85]
 [88 91 56 55]]
```

```
In [89]: print("\n--- Index Location Functions ---")
# 5. Find the *index position* of the min and max values
# (Flattens the tensor if no axis is specified)
print("Index of global Max:")
print(tf.argmax(tf.reshape(tensor, [-1])).numpy())
print("Index of Max in each column:")
print(tf.argmax(tensor, axis=0).numpy())
```

--- Index Location Functions ---

Index of global Max:

3

Index of Max in each column:

```
[[2 2 2 0]
 [3 2 2 0]
 [2 0 2 1]
 [2 3 1 0]]
```

These are all methods and functions that you can use to aggregate, analyze, and manipulate tensors in tensorflow.

Built in tensor functions

You remember we used `+`, `-`, `*`, `/` operators. In tensorflow we can use built in functions that can do exactly the same thing.

```
In [90]: tf.add(tensor, 100)
```

```
Out[90]: <tf.Tensor: shape=(4, 4, 4), dtype=int64, numpy=
array([[[144, 165, 168, 199],
        [137, 163, 158, 196],
        [126, 129, 150, 138],
        [102, 138, 167, 193]],

       [[100, 103, 142, 129],
        [108, 129, 108, 152],
        [183, 114, 157, 199],
        [138, 130, 172, 116]],

       [[176, 194, 189, 185],
        [174, 185, 167, 127],
        [196, 110, 176, 112],
        [158, 115, 163, 179]],

       [[130, 155, 153, 154],
        [188, 134, 125, 155],
        [140, 102, 156, 144],
        [157, 191, 106, 114]])])>
```

```
In [92]: # tf.subtract(tensor, 100)
# tf.multiply(tensor, 100)
# tf.divide(tensor, 100)
```

Also, for matrix multiplication we can use `tf.matmul(tensor1, tensor2)`.

There are various ways you can use tensors in tensorflow but these are the most common ones.

And that's it.

Just one last time,

If you know Numpy, then you know almost everything about Tensorflow.

Final words

This is just the beginning we will do a lot more and learn a lot more as we progress through this article series. So, I hope you guys like it...

HAPPY DEEP LEARNING!!